

Regression am Bsp. Boston house-price data

Nahezu alle Aufgabenstellungen der letzten Wochen dienten dazu, Eingaben in die eine oder Kategorie einzuordnen, zu klassifizieren. Ziel war es, vorherzusagen, ob jemand beispielsweise an Diabetes leidet oder um welche Ziffer es sich handelt. *Machine Learning*-Algorithmen können aber nicht nur Klassifizierungsaufgaben lösen. Ebenso ist es möglich, Hauspreise oder die morgige Temperatur – also kontinuierliche Werte – vorherzusagen. Hierbei handelt es sich um sog. Regressionsprobleme.

Hinweis: Regression ist nicht gleich Regression! Man darf die hier vorgestellte Regression nicht mit der *Logistischen Regression* verwechseln. Das mag jetzt ein wenig verwirrend sein, aber bei der *Logistischen Regression* handelt es sich um einen Klassifizierungsalgorithmus.

Diese Übung beschäftigt sich mit der Vorhersage des Kaufpreises für Häuser. Grundlage bildet das *Boston house-price Dataset*¹. Dieses steht ebenfalls via Keras² zur Verfügung.

The Boston house-price data of Harrison, D. and Rubinfeld, D.L. 'Hedonic prices and the demand for clean air', J. Environ. Economics & Management, vol.5, 81-102, 1978. Used in Belsley, Kuh & Welsch, 'Regression diagnostics ...', Wiley, 1980. N.B. Various transformations are used in the table on pages 244-261 of the latter.

Variables in order:

CRIM	per capita crime rate by town
ZN	proportion of residential land zoned for lots over 25,000 sq.ft.
INDUS	proportion of non-retail business acres per town
CHAS	Charles River dummy variable (= 1 if tract bounds river; 0 otherwise)
NOX	nitric oxides concentration (parts per 10 million)
RM	average number of rooms per dwelling
AGE	proportion of owner-occupied units built prior to 1940
DIS	weighted distances to five Boston employment centres
RAD	index of accessibility to radial highways
TAX	full-value property-tax rate per \$10,000
PTRATIO	pupil-teacher ratio by town
B	$1000(B_k - 0.63)^2$ where B_k is the proportion of blacks by town
LSTAT	% lower status of the population
MEDV	Median value of owner-occupied homes in \$1000's

 19.1 Laden Sie das Dataset und verschaffen Sie sich einen Überblick (z.B. Anzahl der Datensätze, Bedeutung der einzelnen Features (Spalten) etc.). Über wie viele Datensätze verfügt das Dataset insgesamt?

Netzwerkarchitektur für das Regressionsproblem:

```
model = Sequential()
model.add(Dense(32, activation="relu", input_shape=(train_data.shape[1],)))
```

¹ <http://lib.stat.cmu.edu/datasets/boston>

² https://keras.io/api/datasets/boston_housing/

```
model.add(Dense(16, activation="relu"))
model.add(Dense(1))

model.compile(optimizer="adam", loss="mse", metrics=["mae"])
```

Als Aktivierungsfunktion kommt bei den ersten zwei Layern die *ReLU* zum Einsatz. Nachdem wir die Preisvorhersagen auf keinen Fall irgendwie begrenzen möchten, braucht es keine diesbezüglichen Angaben beim letzten Layer. Die Default-Aktivierungsfunktion ist *linear* – dies ist die Option der Wahl, wenn es darum geht, eine kontinuierliche Variable vorherzusagen. Die lineare Aktivierungsfunktion gibt z direkt aus. $\hat{y}$ ist somit ein beliebiger numerischer Wert und nicht eine Wahrscheinlichkeit zw. 0 und 1, wie das bei der *Sigmoid*- oder *Softmax*-Aktivierungsfunktion der Fall wäre.

Als Verlustfunktion ist bei der Kompilierung der mittlere quadratische Fehler (*Mean Squared Error, MSE*) anzugeben. Hierbei handelt es sich um das Quadrat der Differenz zwischen Vorhersage ($\hat{y}$) und Zielvariable (y). Auch in diesem Fall ist festzuhalten, dass diese Verlustfunktion typisch ist bei Regressionsaufgaben.

Als Kennzahl beim Training dient jetzt nicht die bereits bekannte *Accuracy*, sondern der mittlere absolute Fehler (*Mean Absolute Error, mae*). Hierbei handelt es sich um den Betrag der Differenz von Vorhersagen und Zielwerten. Angenommen, der Wert beträgt 1,5 oder 0,5, würde das bei dieser Aufgabe bedeuten, dass die Vorhersagen durchschnittlich um 1500\$ oder 500\$ von den tatsächlichen Werten abweichen.

 **19.1** Trainieren Sie das Modell und evaluieren Sie dieses mit `evaluate`. Anm.: Es hat sich gezeigt, dass rund 32 Epochen für ein brauchbares Ergebnis notwendig sind. Als Batch-Größe ist 8 zu wählen. Um die Ausgabe n-vieler Traces im Notebook zu unterbinden, empfiehlt sich `verbose=0` beim Trainieren. Dokumentieren Sie die den *mae*.

Der Output von `evaluate` ist wie folgt zu verstehen:

```
model.evaluate(X_valid, y_valid)
102/102 [=====] - 0s 460us/step
[18.09196606804343, 2.506760358810425]
```

Die Vorhersagen weichen im Durchschnitt um ca. 2500\$ von den tatsächlichen Werten ab.

Standardisierung und Normalisierung der Daten

Füttert man das Neuronale Netze mit Werten völlig unterschiedlicher Größenordnungen (siehe Abbildung 2), führt das in der Regel zu Problemen beim Lernen. Wie bei den Klassifizierungsaufgaben ist es daher wichtig, die Daten aufzubereiten³.

³ <https://github.com/cs231n/cs231n.github.io/blob/7f027e51ca52a4640f79a598f09eda917c78fc3b/neural-networks-2.md#datapre>

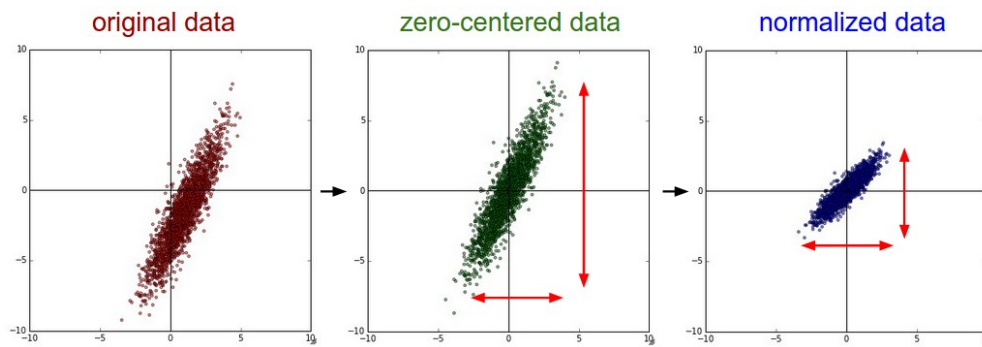


Abbildung 1 Data Standardization and Normalization Pipeline³

Folgende Standardisierungs- und Normalisierungsmaßnahmen können als übliche Praxis angesehen (bei Trainings- und Testdaten) werden:

- a) Spaltenmittelwert bilden und von jedem Wert subtrahieren,
- b) sämtliche Werte einer Spalte durch die Standardabweichung dividieren.

Nach den Maßnahmen zeigt sich, dass die Merkmalswerte um 0 herum zentriert sind und einen annähernd gleichen Wertebereich aufweisen.

WICHTIG: Die Standardisierung der Testdaten hat mit dem Mittelwert und der Standardabweichung der Trainingsdaten zu erfolgen!

Tipp: Das lässt sich mit Numpy ganz leicht erledigen. Es braucht definitiv keinen iterativen Vorgang – das macht Numpy im „Hintergrund“.

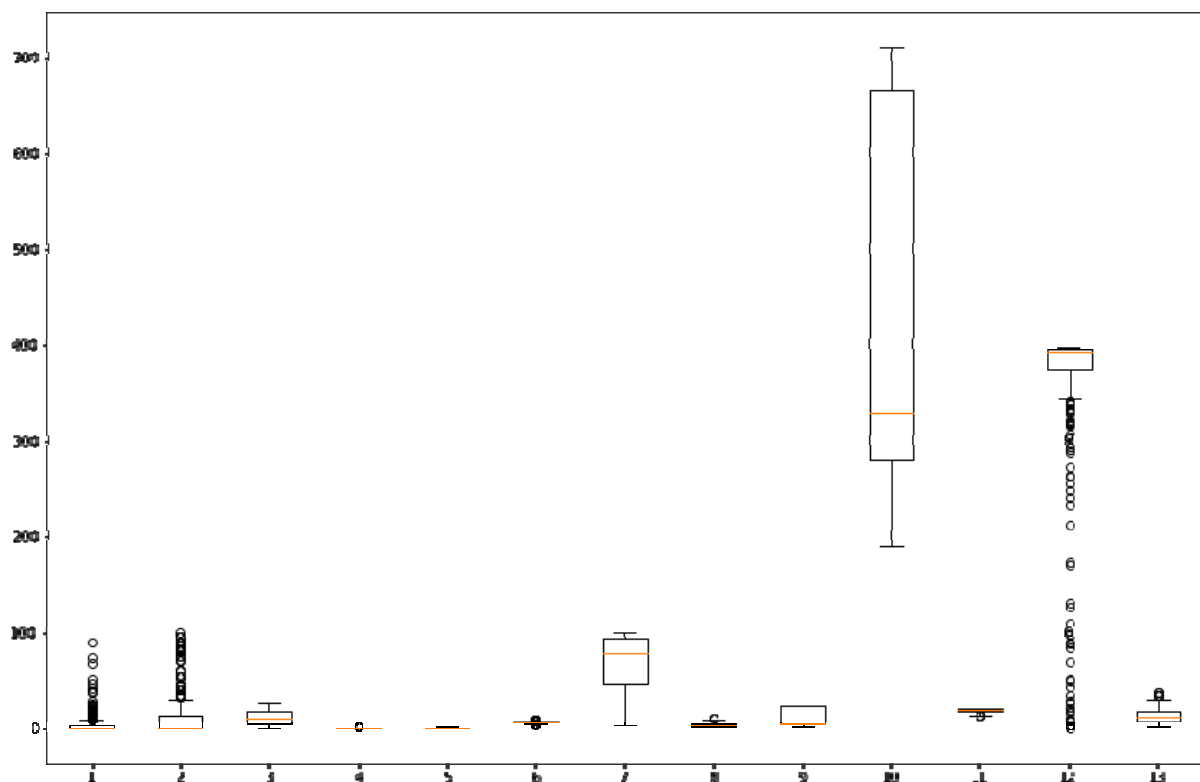


Abbildung 2: Daten vor der Normalisierung

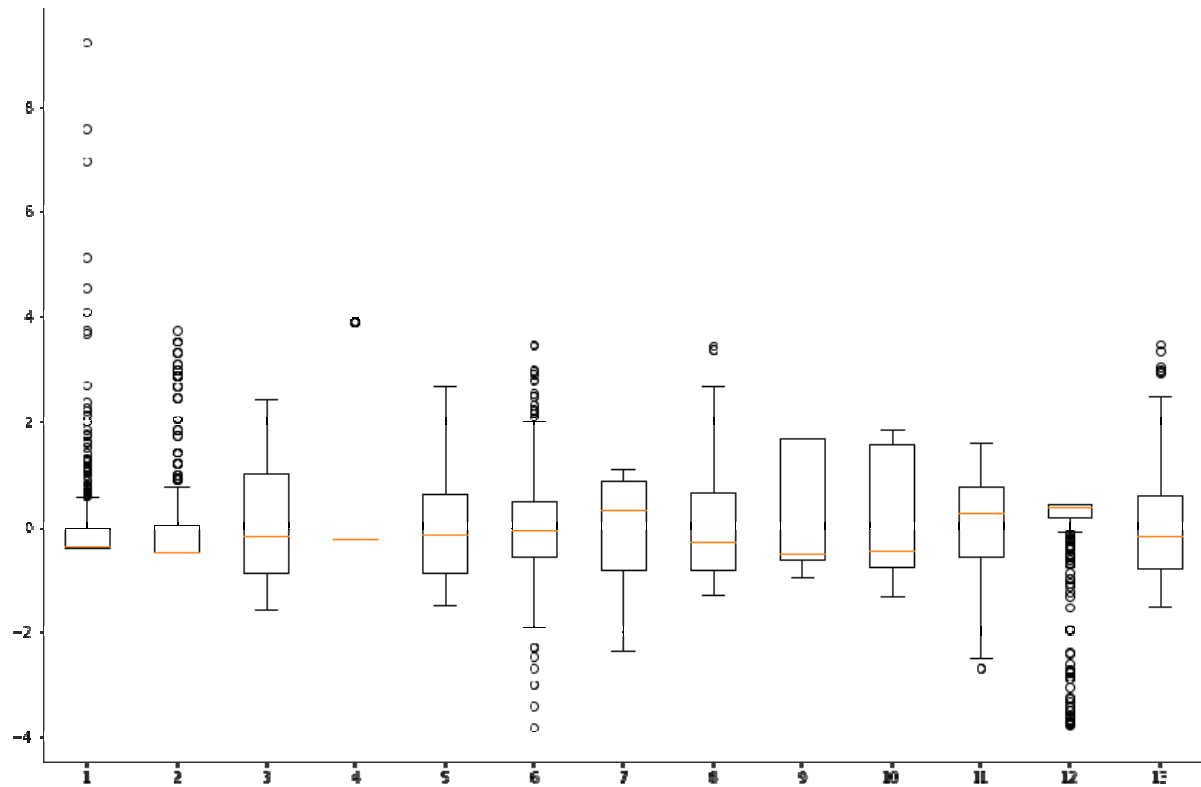


Abbildung 3: Daten nach der Standardisierung

Ausreißer sind hier und jetzt einmal kein Thema!